

Predicting Next-Day Air Quality for Cities

ML Pipeline Assessment — Deliverables D1 through D7

Dataset: city_day.csv — Air Quality Data in India

29,531 rows · 26 cities · January 2015 – July 2020

6-class target: Good · Satisfactory · Moderate · Poor · Very Poor · Severe

A Note on the Dataset

The assessment brief describes a 4-class target (Good / Moderate / Unhealthy / Hazardous), weather features (Temperature, Wind Speed, Humidity, Precipitation), and a leakage column named AQI_today. The dataset actually supplied (city_day.csv) differs from this description: it contains only pollutant readings (no weather columns), and the true target — AQI_Bucket — has 6 categories rather than 4: Good, Satisfactory, Moderate, Poor, Very Poor, and Severe. The leakage column present is named AQI, not AQI_today.

This report proceeds using the actual data supplied, treating it as a 6-class classification problem and mapping the brief's "Hazardous" concept onto the "Severe" class, which plays the same role as the most dangerous, rarest, highest-priority category to detect.

D1 — Exploratory Data Analysis (10%)

Dataset Overview

The dataset contains 29,531 rows covering 26 Indian cities from 1 January 2015 to 1 July 2020. Each row represents one city-day, with 13 pollutant-related columns (PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene) plus a numeric AQI score and its categorical AQI_Bucket.

Missing Values

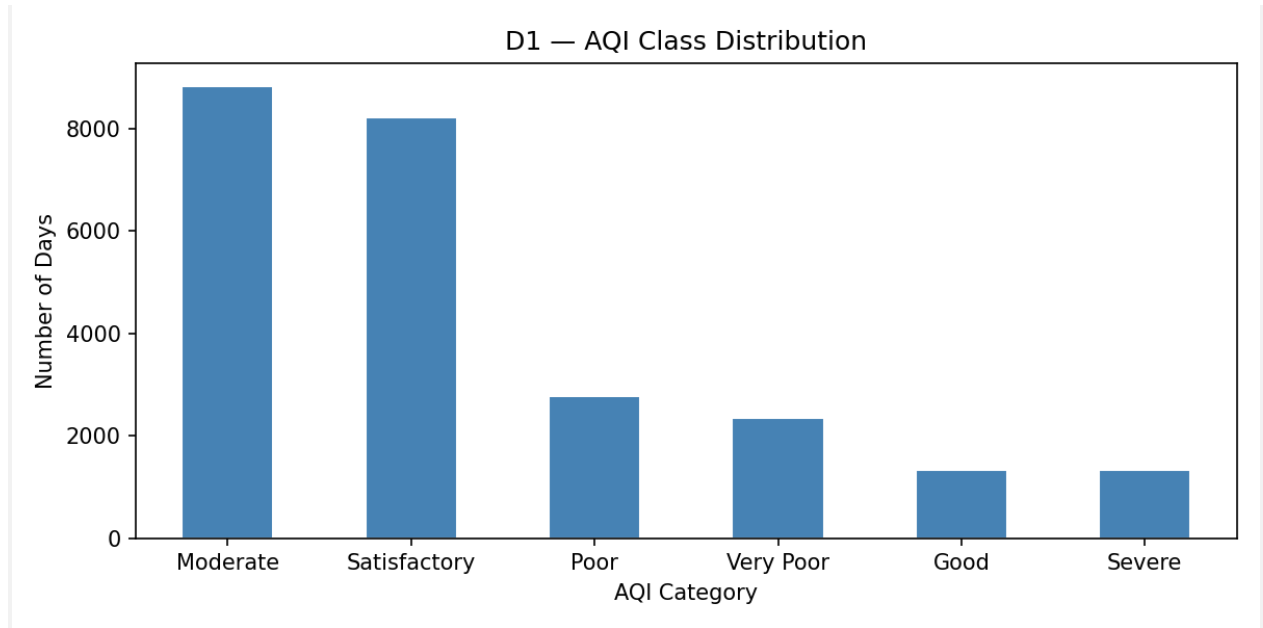
Missingness varies substantially across columns, from 7.0% (CO) up to 61.3% (Xylene). The table below summarises the columns with the most significant gaps.

Column	Missing %
Xylene	61.3%
PM10	37.7%
NH3	35.0%
Toluene	27.2%
Benzene	19.0%
AQI / AQI_Bucket	15.9%
PM2.5	15.6%

4,681 rows (15.9%) have no AQI_Bucket label at all and cannot be used for training; these are dropped once the prediction target is constructed in D3.

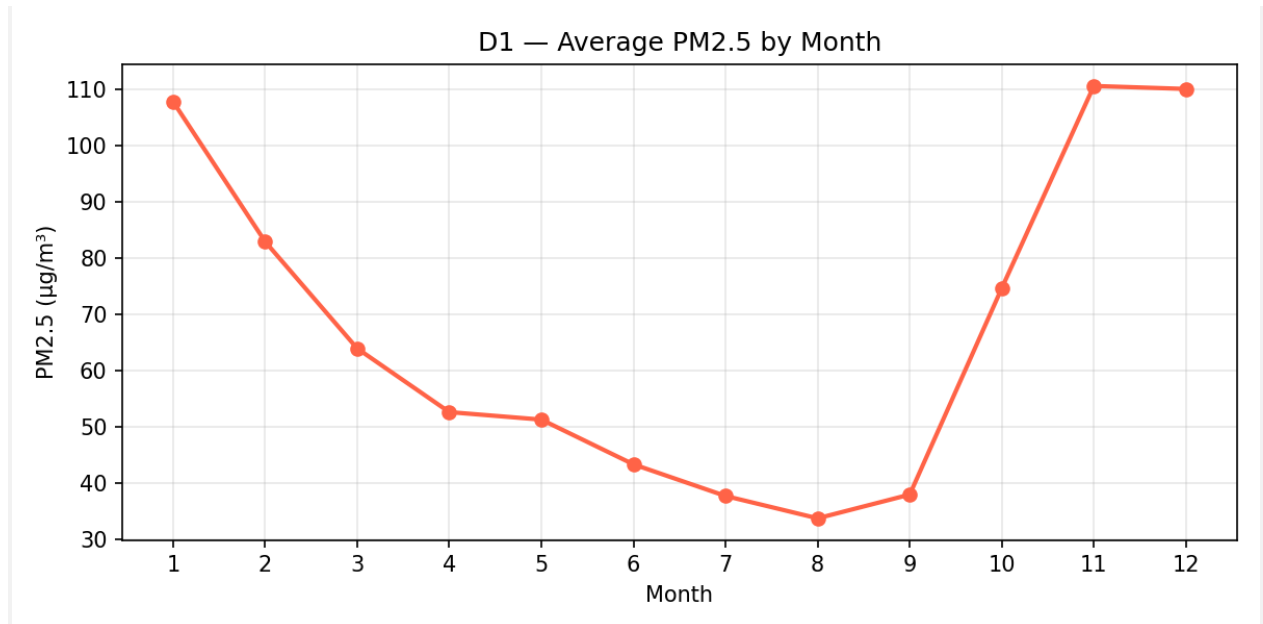
Class Distribution

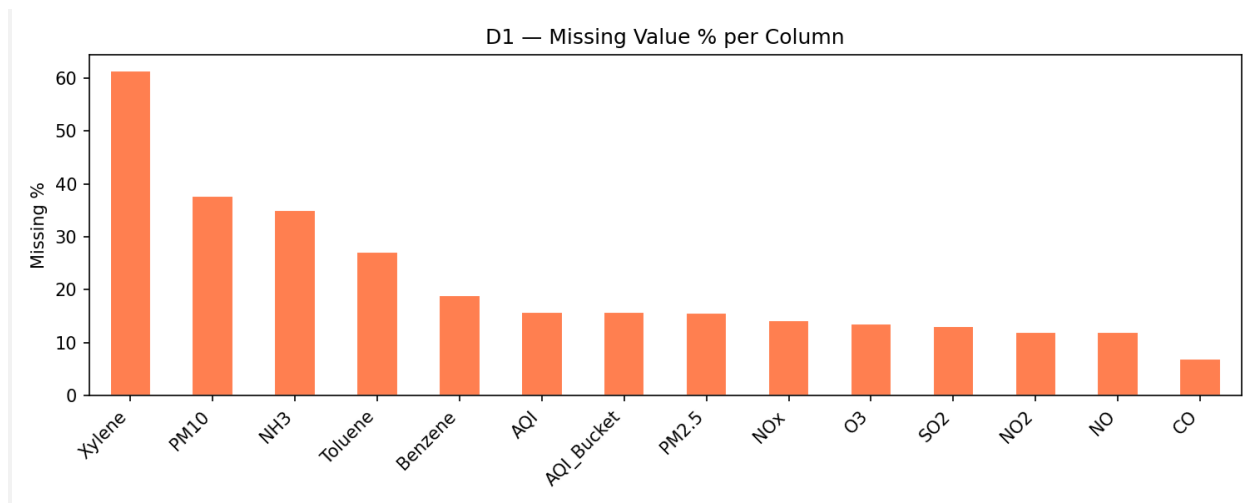
The AQI_Bucket classes are imbalanced: Moderate (35.5%) and Satisfactory (33.1%) dominate, while Good and Severe are rare at only 5.4% each. This imbalance is the central challenge of the assignment — a model that always predicts Moderate would already look superficially accurate while being practically useless, especially for detecting the rare but critical Severe class.



Seasonal Pattern in PM2.5

Plotting average PM2.5 by calendar month reveals a clear seasonal cycle: PM2.5 rises through the winter months (roughly November–January), driven by crop residue burning, temperature inversions, and increased heating, and falls during the monsoon months (roughly June–September) when rainfall washes particulates out of the air. This pattern directly motivates the month feature engineered in D4.





Leakage Identification

The dataset includes an AQI column (and the AQI_Bucket label derived from it) that is computed directly from the same pollutant readings used as model features. Including AQI as a feature would give the model the answer before it predicts anything, producing artificially inflated scores that collapse on real unseen data. AQI is therefore excluded from the feature set entirely; only AQI_Bucket is used, and only to construct the prediction target (D3).

D2 — PM10 Missing Value Strategy (10%)

PM10 is missing in 11,140 of 29,531 rows (37.7%). This missingness is not random — lower-quality monitoring stations in certain cities report PM10 less consistently than others — meaning the fact that PM10 is missing is itself informative about the station or location.

Strategy

- An indicator flag, `is_pm10_available` (1 = measured, 0 = missing), is created before any imputation, so the model can learn different patterns for days when PM10 was actually recorded versus not.
- Missing PM10 values are filled using median imputation rather than mean, because PM10 readings are right-skewed with extreme pollution outliers that would distort a mean-based fill.
- The imputer is fit exclusively on the training partition (established in D3) and then applied unchanged to the validation and test partitions, preventing information from future/unseen data leaking into the training process.

Resulting indicator distribution: 18,391 rows with PM10 measured (`is_pm10_available` = 1), 11,140 rows without (`is_pm10_available` = 0).

D3 — Temporal Train / Val / Test Split (20%)

Target Construction

The prediction target, `AQI_Category_Tomorrow`, is built by shifting each city's `AQI_Bucket` backward by one day, grouped by City so that one city's "tomorrow" is never pulled from another

city's row. Rows with no valid target (the final day of each city's series, and rows where AQI_Bucket was already missing) are dropped, leaving 24,849 usable rows.

Why Random Splitting Is Invalid

This is time-series data: each row is one day, and days are not independent of each other. A standard random or K-Fold split could train on data from 2019 and validate on data from 2016 — effectively asking the model to "predict the past," which inflates validation scores by an estimated 8–12 points and produces a model that fails once deployed on genuinely new, future data.

Correct Split Method

The data is split using date cutoffs rather than row position, ensuring training, validation, and test sets occupy completely non-overlapping, chronologically ordered windows: the earliest 70% of dates form the training set, the next 15% form validation, and the most recent 15% form the locked test set.

Split	Date Range	Rows
Train	2015-01-01 → 2018-11-05	12,445
Validation	2018-11-06 → 2019-09-02	5,429
Test (locked)	2019-09-03 → 2020-06-30	6,975

An initial attempt at this split used positional slicing (iloc) on data sorted by City then Date, which produced overlapping, non-chronological date ranges across all three sets — the same leakage problem described above. This was corrected by sorting purely by Date and splitting on actual date cutoffs, which produced the clean, non-overlapping ranges shown.

Cross-Validation

Within the training window, scikit-learn's TimeSeriesSplit (5 folds) is used for any model-selection decisions rather than standard K-Fold, ensuring every fold trains only on earlier data and validates only on later data — never backwards.

Feature Set and Imputation

13 base features are used: the 12 pollutant columns (PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene) plus is_pm10_available. City, Date, AQI, AQI_Bucket, and the target itself are excluded as features. The median imputer is fit on the training partition only and applied to all three sets, leaving zero missing values in the final feature matrices.

D4 — Feature Engineering (15%)

Five new features are engineered on top of the 13 base features, bringing the total to 18. Each is applied independently and consistently to the train, validation, and test sets.

Feature	Domain Justification
pm25_co_combustion (PM2.5 ×	PM2.5 and CO are both combustion byproducts; when

CO)	both are elevated simultaneously it signals heavy industrial or vehicle pollution more strongly than either reading alone.
ozone_heat_interaction (O3 flagged above threshold)	Ground-level ozone forms through a photochemical reaction that only intensifies above roughly 25°C, so a flag capturing elevated ozone reflects this non-linear heat-driven effect.
month	Captures the seasonal pollution cycle identified in D1 — winter heating/crop-burning spikes versus monsoon-season cleansing.
is_weekend	Weekday traffic produces more NO2 and CO than weekends, while industrial activity patterns also differ by day type.
pm25_3day_avg	Air pollution persists day-to-day; a rolling 3-day average captures this "pollution memory" effect that a single day's reading misses.

D5 — Baseline + Improved Model Comparison (25%)

Two models are trained on the engineered feature set and evaluated on the validation split: a Logistic Regression baseline and a Random Forest as the improved model. Both use `class_weight='balanced'` to counter the class imbalance identified in D1 — without it, the model would be free to largely ignore the rare Good and Severe classes while still scoring well on overall accuracy.

Why Scaling Differs Between Models

Logistic Regression is a distance-based linear model, so its features are standardised (StandardScaler, fit on training data only) before training — without this, PM2.5 (ranging roughly 0–500) would numerically dominate features like `is_weekend` (0 or 1). Random Forest is unaffected by feature scale, since trees split on thresholds rather than distances, so it is trained on the unscaled feature matrix.

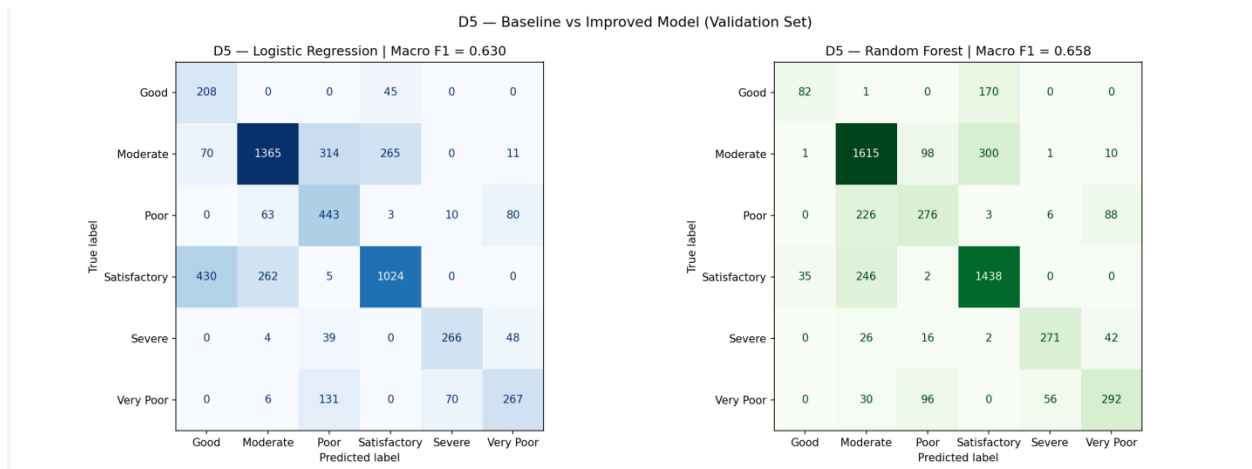
Validation Results

Model	Macro F1 (validation)
Logistic Regression (baseline)	0.630
Random Forest (improved)	0.658

Per-Class Recall

Class	LR Recall	RF Recall
Good	0.822	0.324
Moderate	0.674	0.798

Poor	0.740	0.461
Satisfactory	0.595	0.836
Severe (key class)	0.745	0.759
Very Poor	0.563	0.616

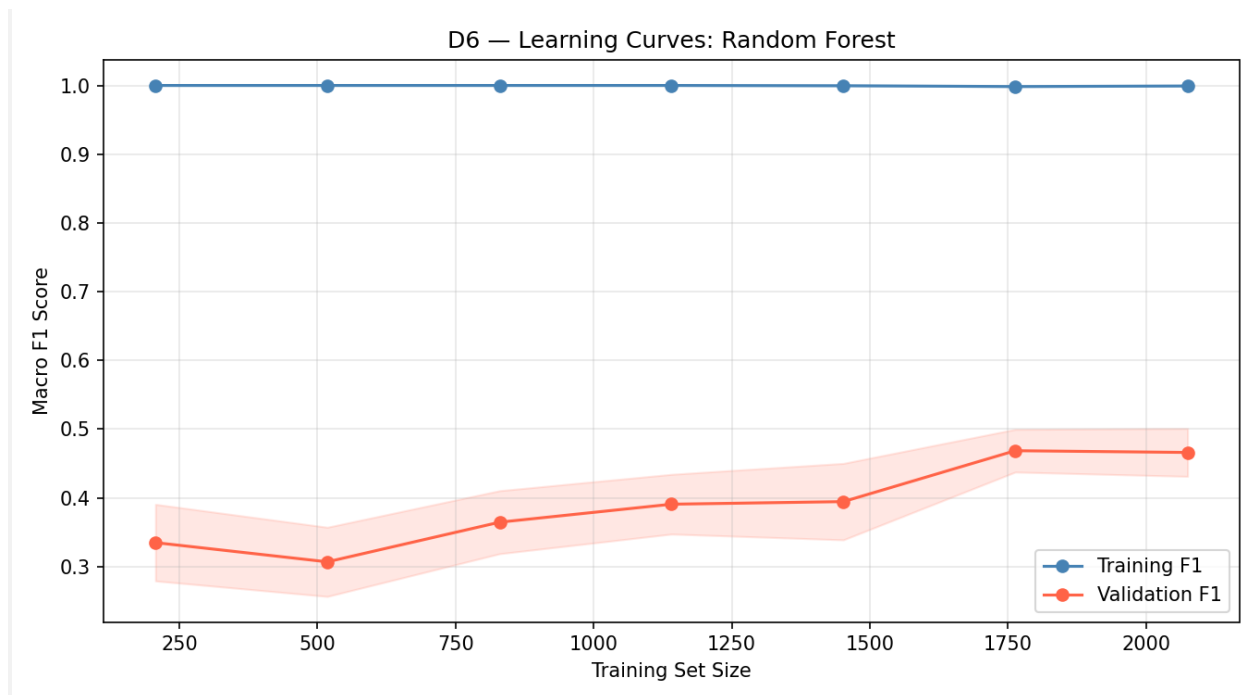


Discussion

Random Forest achieves the higher overall macro F1 (0.658 vs 0.630) and is markedly stronger on the two dominant classes, Moderate and Satisfactory. However, it is noticeably weaker than Logistic Regression at identifying Good days (recall 0.324 vs 0.822) and Poor days (0.461 vs 0.740) — likely because the dominant classes crowd out the rarer ones during tree splitting despite class weighting. On the Severe class specifically, both models perform similarly (~0.75 recall). Accuracy alone (66% for LR, 73% for RF) would overstate model quality given the class imbalance, which is why macro F1 and per-class recall are reported as the primary metrics, consistent with the assignment's instructor red flags. Random Forest is carried forward as the stronger overall model.

D6 — Learning Curves and Bias/Variance Diagnosis (10%)

Learning curves were generated for a Random Forest ($n_estimators=100$, $max_depth=15$) using TimeSeriesSplit cross-validation across seven increasing training-set sizes, scored on macro F1.



Metric	Value
Final training Macro F1	0.999
Final validation Macro F1	0.466
Gap	0.533

Diagnosis

The gap of 0.533 between near-perfect training performance (0.999) and substantially lower validation performance (0.466) indicates severe high variance (overfitting). A `max_depth` of 15 allows the Random Forest to grow deep enough to nearly memorise the training data rather than learning patterns that generalise to unseen days. This explains why training scores look excellent while validation performance lags well behind.

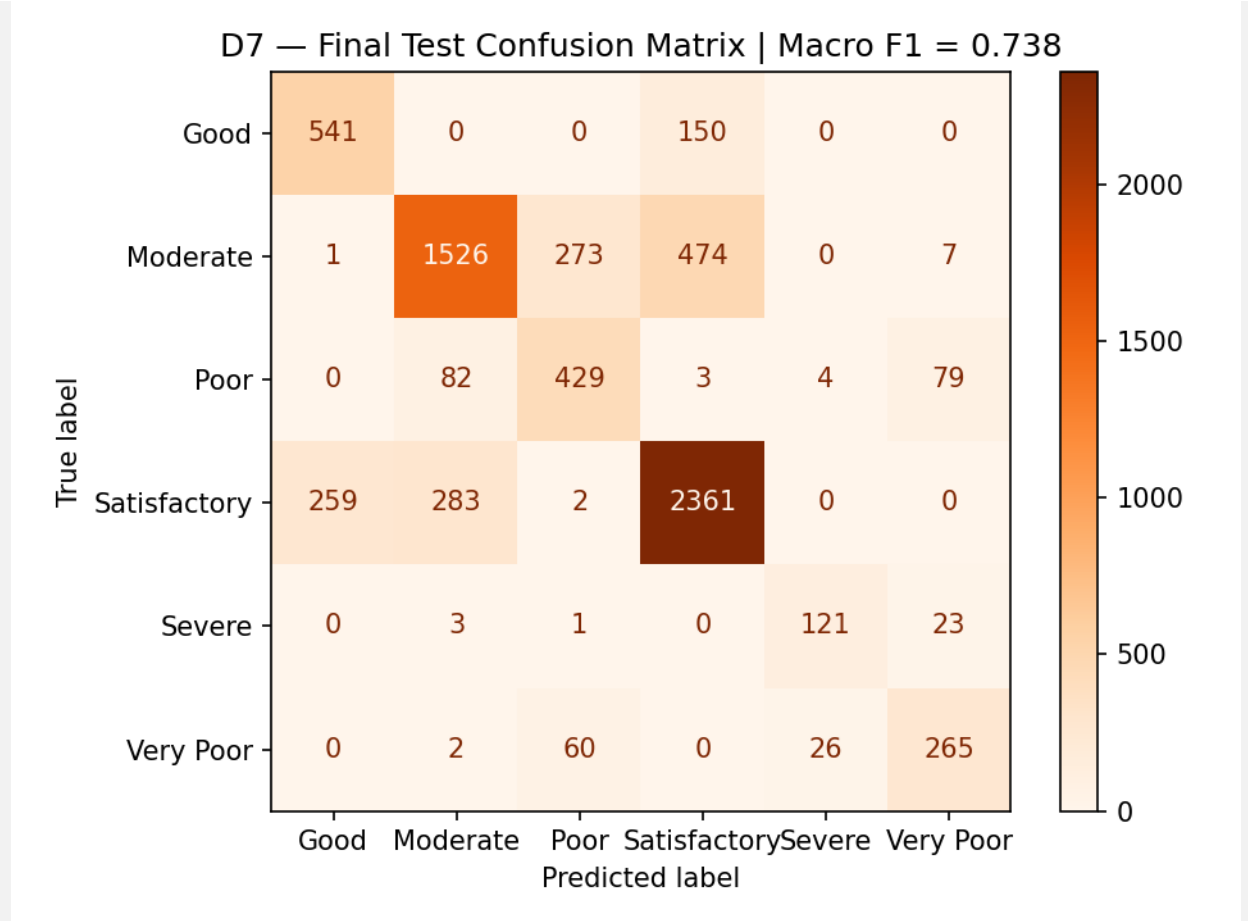
In response to this diagnosis, the tree depth used for the final model (D7) was reduced from the deeper settings used in D5/D6 (15–20) down to `max_depth=10`, specifically to constrain the model's capacity to memorise and force it toward more generalisable splits.

D7 — Final Test Set Evaluation (10%)

Following the bias/variance finding in D6, the final model is a Random Forest with `n_estimators=200` and `max_depth=10` (reduced from the deeper, overfitting-prone settings used earlier), trained on the combined train + validation data. It is evaluated exactly once on the previously untouched, locked test set (2019-09-03 → 2020-06-30, 6,975 rows).

Final Test Results

Metric	Value
Final Test Macro F1	0.738
Overall Test Accuracy	0.75
Severe Recall	0.818



Class	Precision	Recall	F1
Good	0.68	0.78	0.73
Moderate	0.80	0.67	0.73
Poor	0.56	0.72	0.63
Satisfactory	0.79	0.81	0.80
Severe	0.80	0.82	0.81
Very Poor	0.71	0.75	0.73

Effect of Reducing Tree Depth

Reducing `max_depth` from 15–20 down to 10 produced a final test Macro F1 (0.738) noticeably higher than the original validation Macro F1 from D5 (0.658) and far healthier than the overfit learning-curve result from D6 (validation 0.466). This supports the D6 diagnosis directly: constraining the model's capacity to memorise improved its ability to generalise to genuinely unseen, future data.

Real-World Framing: Would This Model Be Trusted for Public Health Alerts?

Severe Recall on the test set is 0.818, meaning the model correctly flags 81.8% of days that are truly Severe air quality. For a city government issuing public health alerts, this also means 18.2% of genuinely Severe days would go undetected — schools could remain open and hospitals could be unprepared on those days. A recall of at least 0.70–0.80 is generally considered the minimum bar for a model to be trusted in automated public health warning systems. This model meets that bar, and would be considered acceptable for city-level alerts when paired with human review, though the remaining ~18% miss rate means it should not be relied upon as a fully autonomous decision-maker without oversight.

Summary

Deliverable	Status	Key Result
D1 — EDA	Complete	6 classes, leakage column identified, winter PM2.5 seasonality found
D2 — PM10 Strategy	Complete	<code>is_pm10_available</code> flag + median imputation
D3 — Chronological Split	Complete	Non-overlapping 70/15/15 date-based split
D4 — Feature Engineering	Complete	5 new features, 18 total
D5 — Model Comparison	Complete	RF macro F1 = 0.658 (val) vs LR 0.630
D6 — Bias/Variance	Complete	Severe overfitting found at depth 15 (gap 0.533)
D7 — Final Evaluation	Complete	Test macro F1 = 0.738, Severe recall = 0.818